Книга 3

Архитектура нейросетей и трансформеров

Нейрон, обратное распространение, механизм внимания, токенизация, RLHF, законы масштабирования

Средний — Продвинутый

Библиотека ИИ · Книга 3 из 5

Содержание

1. 1. Математика одного нейрона
2. 2. Активационные функции
3. 3. Прямой проход (forward pass)
4. 4. Функции потерь
5. 5. Обратное распространение ошибки (backpropagation)
6. 6. Оптимизаторы: SGD, Adam, AdamW
7. 7. Токенизация: BPE, WordPiece, SentencePiece
8. 8. Embeddings: векторы слов и предложений
9. 9. Механизм внимания (Self-Attention)
10. 10. Архитектура трансформера
11. 11. Предобучение: MLM и CLM
12. 12. RLHF и DPO: настройка по человеческим оценкам
13. 13. Законы масштабирования (Scaling Laws)
14. 14. Современные архитектуры: Mixture-of-Experts, Flash Attention

1

Математика одного нейрона

Линейная комбинация + нелинейность

Нейрон — минимальный вычислительный элемент нейросети. Математически он выполняет две операции: взвешенную сумму входов и применение нелинейной функции активации.

Формула

```
z = w₁x₁ + w₂x₂ + ... + wₙxₙ + b
a = f(z)

где:
xᵢ — входные значения (признаки)
wᵢ — веса (learnable parameters)
b — смещение (bias)
z — линейная комбинация (pre-activation)
f — функция активации (нелинейная)
a — выход нейрона (activation)
```

Векторная форма

```
z = W·x + b
a = f(z)

W — матрица весов [out_dim × in_dim]
x — вектор входов [in_dim]
b — вектор смещений [out_dim]
```

Почему важна нелинейность

Без функции активации (f) нейрон — это просто линейная регрессия. Стек линейных слоёв всё равно даёт линейное преобразование. Нелинейность позволяет сети аппроксимировать произвольные функции (теорема универсальной аппроксимации).

Ключевые выводы

- Нейрон = взвешенная сумма + нелинейная активация
- Векторная форма: a = f(W·x + b)
- Нелинейность критически важна — без неё сеть = линейная модель

2

Активационные функции

ReLU, Sigmoid, Tanh, GELU, SwiGLU

Функция	Формула	Где используется
ReLU	max(0, z)	Скрытые слои в CNN, базовый выбор
Sigmoid	$1 / (1 + e^{-z})$	Бинарная классификация, gated RNN
Tanh	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	RNN, когда нужно [-1, 1]
GELU	$z \cdot \Phi(z)$	Трансформеры (BERT, GPT)
SwiGLU	$\text{Swish}(z) \cdot (zW_2)$	Современные LLM (LLaMA, PaLM)
Softmax	$e^{z_i} / \Sigma e^{z_j}$	Выходной слой классификации

ReLU (Rectified Linear Unit)

```
f(z) = max(0, z)

z > 0 → f(z) = z (пропускает сигнал)
z ≤ 0 → f(z) = 0 (блокирует сигнал)

Плюсы: простая, быстрая, нет проблемы затухающего градиента
для положительных значений
Минусы: «мертвые нейроны» — при z < 0 градиент = 0
```

GELU (Gaussian Error Linear Unit)

Используется в трансформерах. Гладкий аналог ReLU, который даёт небольшое отрицательное значение для $z < 0$, что уменьшает проблему мёртвых нейронов.

$$GELU(z) \approx 0.5z \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (z + 0.044715z^3)))$$

Ключевые выводы

- ReLU — базовый выбор для скрытых слоёв
- GELU — стандарт для трансформеров
- SwiGLU — современный вариант для LLM
- Softmax — для выходного слоя классификации

3 Прямой проход (forward pass)

Как данные проходят через сеть

Прямой проход — это вычисление выхода сети по входным данным. Данные проходят последовательно через все слои.

```
Слой 0 (вход): x = [x₁, x₂, x₃]

Слой 1 (скрытый): z₁ = W₁·x + b₁
a₁ = ReLU(z₁)

Слой 2 (скрытый): z₂ = W₂·a₁ + b₂
a₂ = ReLU(z₂)

Слой 3 (выход): z₃ = W₃·a₂ + b₃
ŷ = softmax(z₃) # вероятности классов
```

Размерности

```
Пример: вход 3 признака, 2 скрытых слоя по 64 нейрона, 10 классов

W₁: [64 × 3] b₁: [64]
W₂: [64 × 64] b₂: [64]
W₃: [10 × 64] b₃: [10]

Всего параметров: 64×3 + 64 + 64×64 + 64 + 10×64 + 10 = 4,522
```

Ключевые выводы

- Forward pass — вычисление выхода сети по входу
- Каждый слой: z = W·a_prev + b, затем a = f(z)
- Количество параметров = сумма размерностей всех весов и смещений

4

Функции потерь

Как измеряется ошибка

Функция потерь (loss function) измеряет, насколько предсказание модели отличается от правильного ответа. Цель обучения — минимизировать среднее значение потерь по всем примерам.

Функция	Формула	Задача
MSE	$\Sigma(\hat{y}_i - y_i)^2 / N$	Регрессия
Cross-Entropy	$-\Sigma\, y_i \log(\hat{y}_i)$	Классификация
Binary CE	$-[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	Бинарная классификация
KL Divergence	$\Sigma\, y_i \log(y_i / \hat{y}_i)$	Распределения вероятностей
Huber Loss	гибрид MSE + MAE	Регрессия с выбросами

Cross-Entropy для классификации

```
L = -Σᵢ yᵢ · log(ŷᵢ)

где:
yᵢ — истинная метка (one-hot): [0, 0, 1, 0, ...]
ŷᵢ — предсказанная вероятность класса i

Если модель уверена в правильном классе (ŷ → 1) → loss → 0
Если модель уверена в неправильном → loss → ∞
```

Perplexity (для языковых моделей)

Perplexity = exp(cross-entropy). Мера того, насколько модель «удивлена» правильным ответом. Чем ниже — тем лучше. PPL = 1 — идеальная модель, PPL = |словарь| — случайная.

Ключевые выводы

- MSE — для регрессии, Cross-Entropy — для классификации
- Cross-Entropy сильно штрафует уверенность в неправильном ответе
- Perplexity = exp(loss) — метрика качества языковых моделей

5 Обратное распространение (backpropagation)

Как градиенты доходят до каждого веса

Backpropagation — алгоритм вычисления градиентов функции потерь по каждому параметру сети. Основан на правиле дифференцирования сложной функции (chain rule).

Chain Rule — основа

```
Если y = f(g(x)), то dy/dx = f'(g(x)) · g'(x)

Для нейросети:
∂L/∂W₃ = ∂L/∂ŷ · ∂ŷ/∂z₃ · ∂z₃/∂W₃

∂L/∂W₂ = ∂L/∂z₃ · ∂z₃/∂a₂ · ∂a₂/∂z₂ · ∂z₂/∂W₂

∂L/∂W₁ = ∂L/∂z₂ · ∂z₂/∂a₁ · ∂a₁/∂z₁ · ∂z₁/∂W₁
```

Алгоритм

1. Forward pass: вычислить все z и a для каждого слоя
2. Вычислить loss L по выходу сети
3. Вычислить градиент loss по выходу: ∂L/∂ŷ
4. Двигаясь от последнего слоя к первому, вычислять градиенты по каждому слою через chain rule
5. Для каждого веса: $\partial L/\partial W_i$ = градиент × выход предыдущего слоя
6. Обновить веса: W = W - η · ∂L/∂W (где η — скорость обучения)

> ⚠ **Затухающий градиент**
>
> В глубоких сетях градиенты могут становиться очень маленькими при прохождении через слои (особенно с sigmoid/tanh). До первых слоёв градиент почти не доходит — они не обучаются. Решения: ReLU, residual connections, batch normalization, gradient clipping.

Ключевые выводы

- Backprop = chain rule для вычисления ∂L/∂W для каждого слоя
- Градиенты идут от выхода к входу (обратный проход)
- Веса обновляются: W = W - η · ∂L/∂W
- Затухающий градиент — проблема глубоких сетей, решается ReLU и residual

6

Оптимизаторы: SGD, Adam, AdamW

Как обновляются веса

Оптимизатор	Идея	Когда использовать
SGD	$W = W - \eta \cdot \nabla L$	Простой, с momentum — эффективен для больших моделей
SGD+Momentum	$v = \beta v + \nabla L$; $W = W - \eta v$	Ускоряет движение в правильном направлении
RMSprop	Адаптивный шаг для каждого параметра	RNN, нестационарные цели
Adam	Momentum + RMSprop вместе	Универсальный, большинство задач
AdamW	Adam + decoupled weight decay	Трансформеры, LLM (стандарт)

Adam — формула

```
m = β₁·m + (1-β₁)·∇L # момент (экспоненциальное среднее градиента)
v = β₂·v + (1-β₂)·(∇L)² # экспоненциальное среднее квадрата градиента
m^ = m / (1-β₁ᵗ) # коррекция смещения
v^ = v / (1-β₂ᵗ)
W = W - η · m^ / (√v^ + ε)

Типичные значения: β₁=0.9, β₂=0.999, ε=1e-8, η=1e-3..1e-5
```

AdamW — почему не просто Adam

В классическом Adam weight decay реализован через L2-регуляризацию в градиенте. Это искажает адаптивные шаги. AdamW разделяет их: адаптивный шаг от Adam + независимый weight decay. Это даёт лучшую обобщающую способность для трансформеров.

Ключевые выводы

- SGD — простой, Adam — универсальный, AdamW — стандарт для LLM
- Adam: momentum + адаптивный шаг на параметр
- AdamW: разделяет weight decay и адаптивный шаг

7 Токенизация: BPE, WordPiece, SentencePiece

Как текст превращается в токены

Токенизация — процесс разбиения текста на токены. Это первый шаг перед подачей текста в модель. От качества токенизации зависит эффективность и многоязычность модели.

BPE (Byte-Pair Encoding)

Самый распространённый метод. Начинает с отдельных символов, затем итеративно объединяет самые частые пары в новые токены.

```
Шаг 0: l o w </w> (символы)
Шаг 1: lo w </w> (l+o → lo, часто встречается)
Шаг 2: low </w> (lo+w → low)
Шаг 3: low</w> (low+</w> → слово)

В итоге: «lower» → [low, er] (2 токена вместо 5 символов)
```

WordPiece

Похож на BPE, но объединяет пары, максимизирующие правдоподобие обучающего корпуса. Используется в BERT. Токены-продолжения начинаются с ## (##ing, ##ed).

SentencePiece

Работает на уровне байтов, не зависит от пробелов. Поддерживает любые языки, включая те, где нет пробелов между словами (китайский, японский, тайский). Используется в LLaMA, T5.

Метод	Модели	Особенность
BPE	GPT-2, GPT-3, GPT-4	Частотное объединение пар
WordPiece	BERT, DistilBERT	Правдоподобие, ##-префикс
SentencePiece	LLaMA, T5, ALBERT	Байтовый уровень, язык-агностик

Ключевые выводы

- BPE — стандарт для GPT-семейства
- WordPiece — для BERT, использует ##-префиксы
- SentencePiece — для многоязычных моделей, работает на уровне байтов

8

Embeddings: векторы слов и предложений

Как слова становятся числами

Embedding — это отображение дискретных токенов в непрерывные векторы. Вектор обучается так, что похожие по смыслу слова оказываются близко друг к другу в векторном пространстве.

Embedding-слой

```
Словарь: V = 50,000 токенов
Размерность: d = 768

Embedding-матрица: E [V × d] = [50000 × 768]

Токен с ID=42 → E[42] → вектор [768 чисел]

Эта матрица — обучаемые параметры модели.
```

Свойства embedding-пространства

```
король - мужчина + женщина ≈ королева
Париж - Франция + Германия ≈ Берлин

cos(вектор(«кошка»), вектор(«собака»)) > cos(вектор(«кошка»), вектор(«стол»))

где cos(a, b) = (a·b) / (|a|·|b|) — косинусное сходство
```

Positional Encoding

Трансформер обрабатывает все токены одновременно, поэтому ему нужно явно указать порядок. Positional encoding добавляет информацию о позиции к embedding-вектору.

```
Синусоидальный (оригинальный трансформер):
PE(pos, 2i) = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

RoPE (Rotary Position Embedding) — современный вариант:
поворачивает embedding-векторы в зависимости от позиции.
Используется в LLaMA, GPT-NeoX, PaLM.
```

Ключевые выводы

- Embedding = таблица V×d, обучается вместе с моделью
- Похожие слова → близкие векторы (cosine similarity)
- Positional encoding добавляет информацию о порядке токенов
- RoPE — современный метод для позиций в LLM

9 Механизм внимания (Self-Attention)

Сердце трансформера

Attention — механизм, позволяющий модели «смотреть» на все токены входа и определять, какие из них важны для каждого выходного токена.

Scaled Dot-Product Attention

```
Attention(Q, K, V) = softmax(Q·Kᵀ / √dₖ) · V

где:
Q (Query) = X · Wq — «что я ищу»
K (Key) = X · Wk — «что у меня есть»
V (Value) = X · Wv — «содержание»
dₖ — размерность ключа (деление на √dₖ стабилизирует градиент)

Q·Kᵀ — матрица «скоролов» [seq_len × seq_len]
softmax — превращает скоры в вероятности (сумма = 1)
× V — взвешенная сумма значений
```

Multi-Head Attention

Вместо одного attention модель использует несколько «голов» (heads), каждая смотрит на разные аспекты отношений между токенами.

```
MultiHead(Q,K,V) = Concat(head₁, head₂, ..., headₕ) · W_O

headᵢ = Attention(Q·Wqᵢ, K·Wkᵢ, V·Wvᵢ)

Пример: d_model=768, h=12 голов → каждая голова d=768/12=64
```

Causal (Masked) Attention

В языковых моделях каждый токен может «видеть» только предыдущие, но не будущие. Это достигается маскированием: будущим позициям присваивается -∞ перед softmax, что даёт вес 0.

```
Mask (для seq_len=4):
[ 0, -∞, -∞, -∞ ]
[ 0, 0, -∞, -∞ ]
[ 0, 0, 0, -∞ ]
[ 0, 0, 0, 0 ]

После softmax: нижний треугольник, верхний = 0
```

Ключевые выводы

- Attention(Q,K,V) = softmax(Q·Kᵀ/√dₖ)·V
- Multi-head: несколько голов параллельно, потом concat
- Causal mask: каждый токен видит только прошлые — для LLM

10

Архитектура трансформера

Сборка блоков в полную модель

Трансформер состоит из повторяющихся блоков. Каждый блок: Multi-Head Attention + Feed-Forward Network, с residual connections и layer normalization.

Блок трансформера (Decoder-only, как GPT)

```
Вход: x [batch, seq_len, d_model]

┌──────────────────────────────────────┐
│ 1. Causal Multi-Head Attention       │
│ x' = MHA(x)                          │
│ x = x + x' (residual)                │
│ x = LayerNorm(x)                     │
├──────────────────────────────────────┤
│ 2. Feed-Forward Network              │
│ x' = Linear(ReLU(Linear(x)))         │
│ x = x + x' (residual)                │
│ x = LayerNorm(x)                     │
└──────────────────────────────────────┘
↑ повторить N раз (слоёв)

Выход: logits = Linear(x) [batch, seq_len, vocab_size]
```

Ключевые гиперпараметры

Параметр	GPT-2 small	GPT-3	LLaMA-2 70B
Слоёв (n_layers)	12	96	80
d_model	768	12288	8192
Голов (n_heads)	12	96	64
Параметров	124M	175B	70B
Контекст	1024	2048	4096

Residual Connections — зачем

Residual (x = x + f(x)) позволяет градиенту проходить напрямую через все слои. Без них глубокие сети (50+ слоёв) не обучаются. Также позволяет модели «пропускать» преобразование, если оно не нужно для данного входа.

Ключевые выводы

- Трансформер = стек из N блоков (Attention + FFN + residual + norm)
- Residual connections критичны для глубоких сетей
- Decoder-only архитектура (GPT) — стандарт для LLM

11 Предобучение: MLM и CLM

Как модели учатся языку

Masked Language Modeling (MLM)

Используется в BERT. Часть токенов маскируется, модель должна предсказать их. Модель видит контекст с обеих сторон (двунаправленная).

```
Вход: «[MASK] сидел на [MASK]»
Цель: предсказать «Кот» и «ковре»
Маскируется ~15% токенов случайно
```

Causal Language Modeling (CLM)

Используется в GPT. Модель предсказывает следующий токен, видя только предыдущие (однонаправленная).

```
Вход: «Кот сидел на»
Цель: «ковре»

Loss = -log P(ковре | Кот, сидел, на)
```

Свойство	MLM (BERT)	CLM (GPT)
Направление	Двунаправленная	Однонаправленная (слева направо)
Задача	Предсказать [MASK]	Предсказать следующий токен
Генерация	Нет	Да (авторегрессионная)
Лучше для	Понимание текста, классификация	Генерация текста, диалоги

Ключевые выводы

- MLM (BERT): маскируем токены, предсказываем их, двунаправленная
- CLM (GPT): предсказываем следующий токен, однонаправленная
- CLM — основа всех генеративных LLM (ChatGPT, Claude, LLaMA)

12

RLHF и DPO: настройка по оценкам людей

От сырой модели к полезной и безопасной

После предобучения модель умеет генерировать текст, но не всегда следует инструкциям и может быть небезопасной. RLHF и DPO — методы дообучения, использующие оценки людей.

RLHF (Reinforcement Learning from Human Feedback)

1. Собрать человеческие оценки: для одного промпта — несколько ответов модели, люди ранжируют их от лучшего к худшему
2. Обучить модель наград (reward model) предсказывать человеческие предпочтения
3. Оптимизировать языковую модель с помощью PPO (Proximal Policy Optimization), максимизируя награду от reward model
4. Контролировать, что модель не «забыла» язык (KL-дивергенция с исходной моделью)

```
Схема RLHF:

Промпт → LLM → 2 ответа → Человек ранжирует
↓
Reward Model
↓
Промпт → LLM → ответ → Reward Model → награда → PPO → обновление LLM
```

DPO (Direct Preference Optimization)

Более новый метод (2023). Обходит reward model и PPO — оптимизирует модель напрямую по парам предпочтений. Проще, стабильнее и дешевле, чем RLHF. Используется в LLaMA-3, Zephyr.

```
DPO loss:
L = -log σ(β · [log π(y_w|x)/π_ref(y_w|x)
- log π(y_l|x)/π_ref(y_l|x)])

где:
y_w — предпочтительный ответ (winner)
y_l — менее предпочтительный (loser)
π — текущая модель, π_ref — исходная
β — параметр регуляризации
```

Ключевые выводы

- RLHF: reward model + PPO, сложнее но проверенный
- DPO: прямая оптимизация по парам, проще и стабильнее
- Оба метода делают модель более полезной и безопасной

13

Законы масштабирования (Scaling Laws)

Как размер модели, данных и вычислений влияют на loss

Исследования OpenAI (2020) и DeepMind (Chinchilla, 2022) показали, что loss модели предсказуемо уменьшается с ростом параметров, данных и вычислений — по степенному закону.

Kaplan et al. (OpenAI, 2020)

```
L(N) ∝ N^(-α)

где:
L — loss (cross-entropy)
N — количество параметров
α ≈ 0.076

Вывод: больше параметров → ниже loss, предсказуемо
```

Chinchilla (DeepMind, 2022)

Показала, что предыдущие модели были «недообучены» — параметров много, а данных мало. Оптимальное соотношение:

```
D_optimal ≈ 20 × N

То есть: для модели с 70B параметров нужно ~1.4T токенов данных

Пример:
GPT-3: 175B параметров, 300B токенов → недообучена
Chinchilla: 70B параметров, 1.4T токенов → лучше при меньшем размере
```

Практическое следствие

Удвоение размера модели без удвоения данных — пустая трата. Бюджет на данные должен расти пропорционально размеру модели. Это изменило подход к обучению LLM в 2022-2024 годах.

Ключевые выводы

- Loss уменьшается по степенному закону с ростом параметров и данных
- Chinchilla: оптимально ~20 токенов на параметр
- Многие большие модели были недообучены — данных было мало

14

Современные архитектуры

Mixture-of-Experts, Flash Attention, KV Cache

Mixture-of-Experts (MoE)

Вместо одной большой FFN — несколько «экспертов» (FFN), и маршрутизатор (router) выбирает 2-4 эксперта для каждого токена. Это позволяет иметь большую общую ёмкость при той же стоимости инференса.

```
Пример: Mixtral 8×7B
8 экспертов по 7B параметров
На каждый токен: 2 эксперта активны
Общих параметров: 46.7B
Активных на токен: 12.9B
→ качество как у 47B модели, скорость как у 13B
```

Flash Attention

Optimized алгоритм attention, который минимизирует чтения/записи в GPU память. Даёт 2-4x ускорение и экономит память, не меняя математический результат.

KV Cache

При авторегрессионной генерации каждый новый токен пересчитывает attention со всеми предыдущими. KV Cache сохраняет Key и Value для предыдущих токенов, чтобы не вычислять их заново.

```
Без KV Cache: генерация N токенов → O(N²) attention вычислений
С KV Cache: генерация N токенов → O(N) attention вычислений

Цена: память GPU растёт линейно с длиной контекста
```

Grouped-Query Attention (GQA)

Используется в LLaMA-2/3. Разделяет Q-головы на группы, использующие общие K и V. Уменьшает размер KV Cache и ускоряет инференс без заметной потери качества.

Ключевые выводы

- MoE: несколько экспертов, роутер выбирает — больше ёмкость при той же скорости
- Flash Attention: 2-4x ускорение, меньше памяти
- KV Cache: O(N²) → O(N) для генерации
- GQA: уменьшает KV Cache, ускоряет инференс

Конец Книги 3. Следующая — Книга 4: Практическое применение ИИ.